Nordic nRF9160

Cellular node: no gateway, 9.4 years

MCU	nRF9160 SiP, M33 @ 64 MHz
Radio	LTE-M + GNSS, integrated modem
Sensor	IIS3DWB on SPI3
RAM	**256 kB** — the binding constraint
Pipeline	101 kB after two optimisations
Battery life	**9.4 y** hourly, D-cell $LiSOCl_2$

Branch `hw/nrf9160-cellular`.

Never flashed. The LTE energy figures are the least certain numbers in this repository — Nordic app-note arithmetic, and real networks vary by operator, band and how the operator configures PSM. A $2\times$ error would move 9.4 years to between 5 and 19.

It is also the only kit that reports a *measured* sample rate.

The most expensive radio has the longest battery life

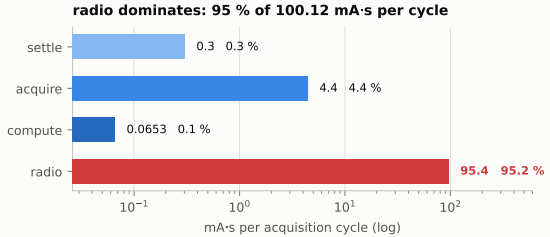
	Radio per TX	Cell	Self-disch.	Life, hourly
STWIN.box, BLE	0.29 mA·s	480 mAh LiPo	20 μA	1.6 y
nRF9160, LTE-M	**95 mA·s**	D $LiSOCl_2$	**4 μA**	**9.4 y**
ESP32-S3, Wi-Fi	240 mA·s	18650 Li-ion	72 μA	1.7 y

330× the radio energy of BLE and 5.8× the life. The mechanism is not the radio:

```
LTE-M pulses 200-250 mA
 -> within a LiSOCl2 D-cell's capability
   -> self-discharge 0.08 %/month = 4 uA
     -> circuit (40 uA) dominates
       -> the design actually matters -> 9.4 y

Wi-Fi bursts 350 mA
 -> outside LiSOCl2's pulse capability
   -> forces Li-ion, 2.5 %/month = 72 uA
     -> cell dominates above hourly

       -> saturates at 2.7 y, firmware irrelevant
```



So what: On a low-duty sensor, battery life is usually a chemistry decision dressed up as a firmware problem. Three of our five battery kits have a cell whose self-discharge exceeds the circuit's draw.

256 kB forced two optimisations every kit should adopt

	Naive	This kit
Raw int16 FIFO staging	49 152	**0**
Window + twiddle + bit-reversal	46 096	**0** (flash)
Acquisition `float[8192]`	32 768	32 768
Envelope `float[8192]`	32 768	32 768
FFT in place	16 384	16 384
Magnitude + spectrum	16 392	16 392
Chunk scratch	—	3 072
Total	**168 kB**	**101 kB**

1. No raw staging buffer. Each 512-sample FIFO chunk is converted to float *as it arrives*, through a 3 kB scratch. Saves 48 kB and costs nothing — the conversion was going to happen anyway. The other kits stage the raw block because they can afford to, which is a weak reason.

2. Tables in flash. 46 kB of a 1 MB part buys 46 kB of a 256 kB part, and removes ∼40 ms of boot transcendentals.

Second-order effect: that boot cost is one-off on a kit that retains SRAM and *per-cycle* on one that does not — so the optimisation is worth most on the ESP32-S3, even though the nRF9160 forced it.

`make -C cdsp test-flash-tables`: 0 of 43 features differ, worst 2.4e-6.

Two radio decisions worth more than any DSP optimisation

PSM, not System OFF

RRC connection (registered)	$\sim$95 mA·s
Full LTE attach	$\sim$1000 mA·s

System OFF reaches $\sim$2 μA against PSM's $\sim$3 μA — and drops the registration, so every wake pays an attach. Saving 3.6 mA·s per hour to spend 905 is a bad trade **by a factor of 250**.

The PSM parameters are *requests*. An operator that grants a 1-hour TAU instead of 10.5 turns a 9-year node into a 2-year one — which is why verifying the granted values with `AT+CEREG?` is on the checklist and is not a formality.

NB-IoT fallback disabled

Better coverage, $\sim$10$\times$ worse latency. A modem that falls back automatically changes the energy budget by an order of magnitude *without telling anyone*, and the symptom is a fleet where some nodes mysteriously have a quarter of the expected life.

If a site needs NB-IoT that is a per-device deployment decision in the asset registry, not something the modem decides at 3 a.m.

Snapshots at most daily

A 51 kB waveform is $\sim$540 s of airtime and $\sim$2 600 mA·s — **27 acquisitions' worth of energy for one waveform.**

The only kit that measures its own sample rate

`k_uptime_get()` runs from the 32.768 kHz LFCLK — a ±20 ppm crystal. Timing a 307 ms acquisition to 1 ms resolution gives `samples / elapsed` to **0.33 %**, against the IIS3DWB oscillator's ±3 % specification.

The STWIN.box *cannot* do this: nothing there times the acquisition against a crystal, so its fix needs a reference tone through the self-test actuator. The ESP32-S3 could and does not yet.

Why 3 % matters, concretely.

A 3 % error computes a 106.3 Hz BPFO line as if it were at 103.1 Hz. The matcher's tolerance is ±2 % with a 1.5-bin floor, so the fundamental still matches.

By the **fifth harmonic** the error is 16 Hz against a ±10.6 Hz window. The harmonic falls outside, and `n_matched` drops from 5 to 3 or 4.

`n_matched` is one of the 63 features the detector was *trained* on. So a kit with a 3 % rate error produces systematically different features from one with 0.3 % — on the same machine.

So what: Implemented here, identified-and-not-done on the ESP32-S3, and needing new hardware on the STWIN.box. Three kits, three answers, all written down.